

Data mining & machine learning

6. Time series forecasting

Francesco Pistolesi

University of Pisa

francesco.pistolesi@unipi.it

M.Sc. in Artificial Intelligence and Data Engineering



UNIVERSITÀ DI PISA

Learning goals

By the end of this lecture, you will be able to:

- decompose a time series into **trend**, **seasonality**, and **residuals**;
- test for **stationarity** with the ADF test and remove it by differencing;
- understand and fit a **SARIMA** model; select its parameters via AIC;
- **validate** a model with residual diagnostics and the Ljung-Box test;
- produce **point forecasts** and **confidence intervals**;
- evaluate with **MAPE**, **backtesting**, and understand nested cross-validation;
- apply **ML regressors** to forecasting via lag features.

Main topics

decomposition

stationarity

SARIMA + AIC

diagnostics + forecasting

evaluation + backtesting

ML for forecasting

Part 1

The domain

What is a time series?

A **time series** is an ordered sequence of observations indexed by time:

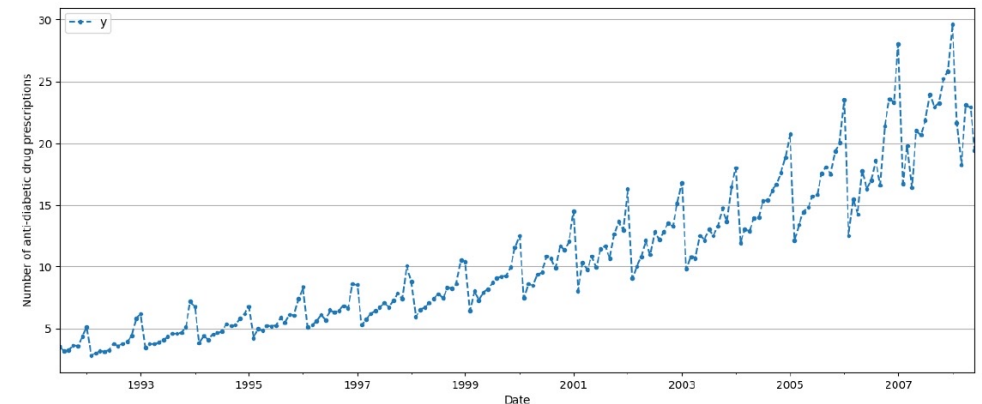
$$y_{t_1}, y_{t_2}, \dots, y_{t_T}$$

Unlike i.i.d. data, **order matters**: each observation depends on previous ones.

Running example

Monthly antidiabetic drug prescriptions in Australia (1991–2008), from the Australian Health Insurance Commission.

Goal: forecast future values $\Rightarrow$ plan production.



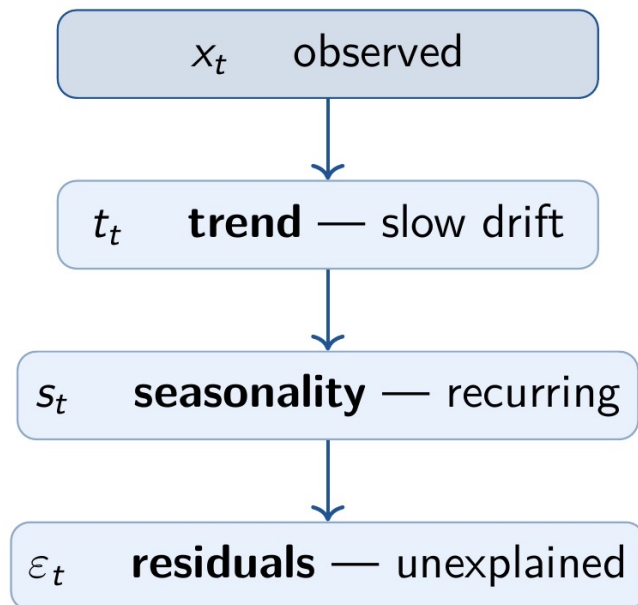
The series grows over time (trend) and oscillates within each year (seasonality).

Two patterns are immediately visible:

- a rising **trend** over the years
- a repeating **seasonal** pattern each year

Time series decomposition

Decomposition breaks a series into three interpretable components.



Additive model

$$x_t = t_t + s_t + \varepsilon_t$$

Use when seasonal fluctuations are **constant** regardless of the level.

Multiplicative model

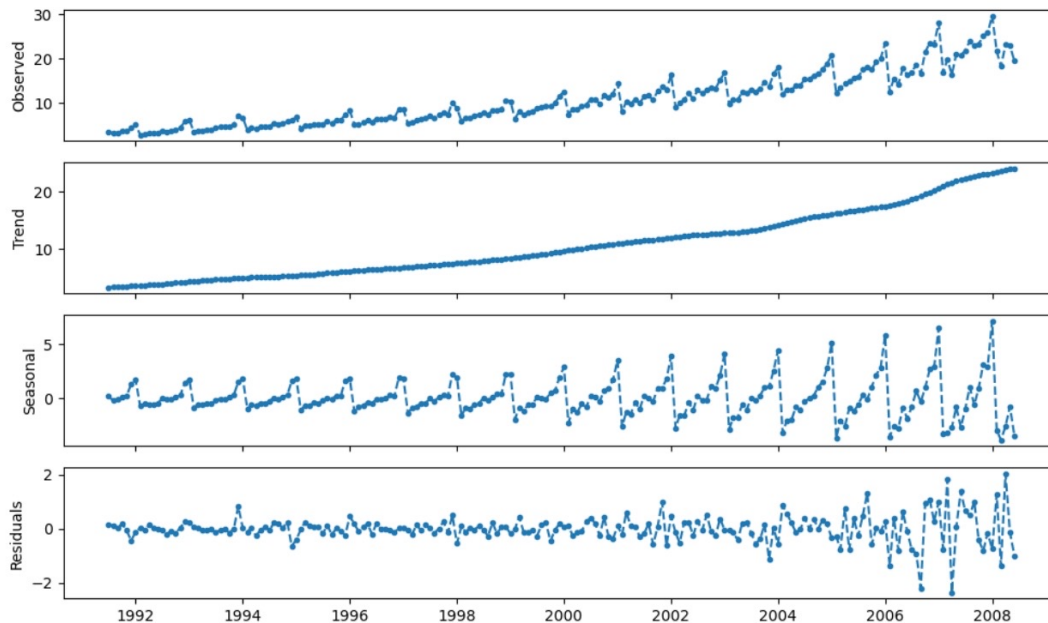
$$x_t = t_t \cdot s_t \cdot \varepsilon_t$$

Use when seasonal fluctuations **grow with the trend** (as in our series).

Equivalent to additive on $\log x_t$.

STL (Seasonal-Trend using Loess): decomposition method that fits trend and seasonal components via local regression; assumes additive form.

Decomposition: additive (STL)



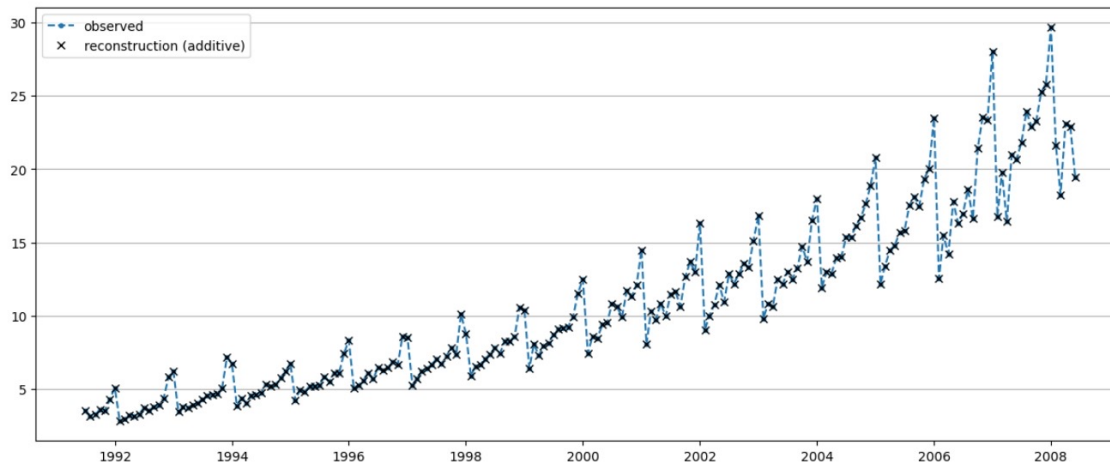
Reading the plot:

- **Observed:** raw series — trend + seasonality + noise.
- **Trend:** smooth long-term growth.
- **Seasonal:** an yearly pattern repeats, but **its amplitude grows** with the trend.
- **Residuals:** what is left over; small but increasing spread in later years.

Note

The seasonal component amplitude is **not constant**, but grows with the trend: the additive model is not the best choice here.

Additive reconstruction: $x_t = t_t + s_t + \varepsilon_t$



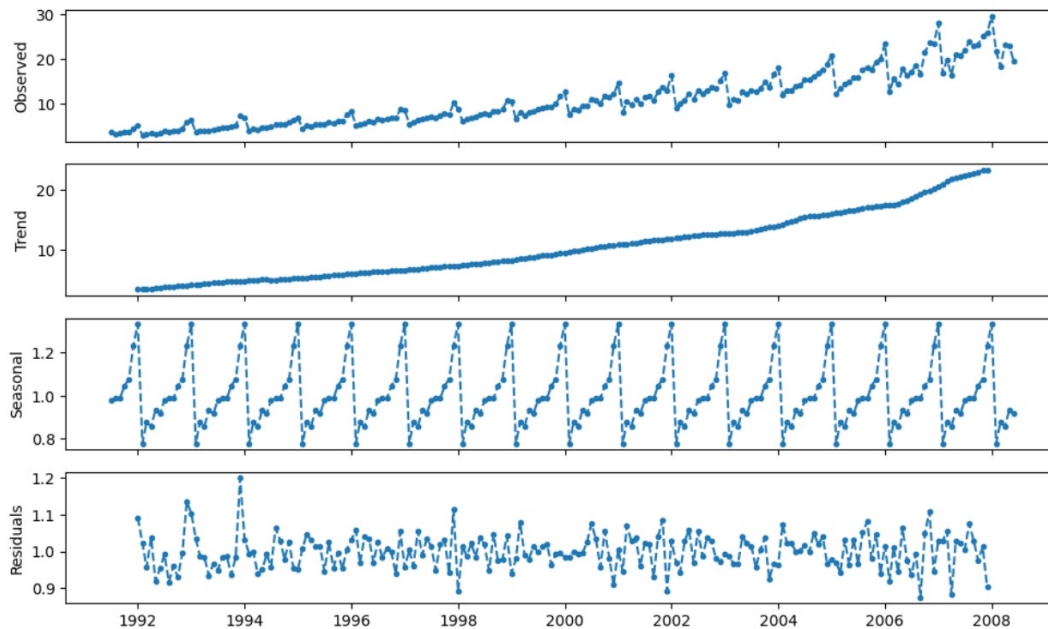
The crosses show $t_t + s_t + \varepsilon_t$ overlaid on the original (dashed).

The reconstruction is **exact**: the three components sum back to the observed series: this is guaranteed by construction.

Why does this matter?

It confirms that the three components are a complete and non-overlapping partition of the signal. Nothing is lost or double-counted.

Multiplicative decomposition: $x_t = t_t \cdot s_t \cdot \varepsilon_t$



Differences from additive:

- **Seasonal component** oscillates around **1.0** (a ratio, not a difference). Values above 1 mean "above-trend month", below 1 "below-trend month".
- **Residuals** oscillate around **1.0** too, and are now much more homogeneous in spread.

Conclusion

The multiplicative model gives **more stable residuals** — a better fit for this series, where amplitudes grow with level.

Stationarity

A time series is **stationary** if its statistical properties do not change over time:

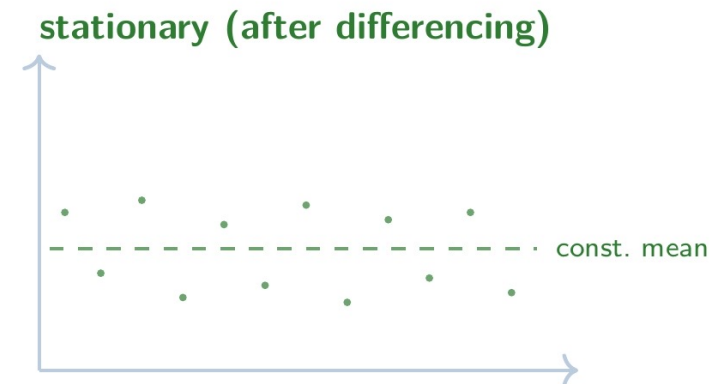
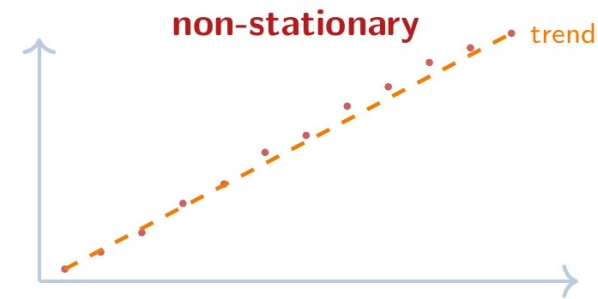
- the **mean** is constant
- the **variance** is constant
- the **covariance** between y_t and y_{t+k} depends only on k , not on t

Why it matters

ARIMA/SARIMA assume stationarity. A series with a trend or growing variance violates this assumption and must be transformed first.

How to check: visual inspection + ADF test.

How to fix: differencing.



ADF test and differencing

Augmented Dickey-Fuller (ADF) test

Tests for the presence of a **unit root**, i.e., a formal indicator of non-stationarity.

H_0 : unit root present $\Rightarrow$ series is **not stationary**

H_1 : no unit root $\Rightarrow$ series **is stationary**

Decision: reject H_0 if $p < 0.05$.

Results on the antidiabetic series:

Series	p -value	Conclusion
Original y_t	1.00	not stationary
After Δy_t	0.12	not stationary
After $\Delta\Delta_{12}y_t$	$\ll 0.05$	stationary!

Differencing

First-order difference (removes trend):

$$y'_t = y_t - y_{t-1} \quad (d = 1)$$

Seasonal difference (removes seasonality):

$$y''_t = y'_t - y'_{t-m} \quad (D = 1, m = 12)$$

The number of regular differences d and seasonal differences D are **fixed parameters** of SARIMA: determined by the ADF test, not by grid search.

ACF and PACF: measuring temporal dependencies

Autocorrelation Function (ACF)

$$\text{ACF}(k) = \text{corr}(y_t, y_{t-k})$$

Measures the **total** correlation between y_t and its value k steps back, including indirect effects through intermediate lags.

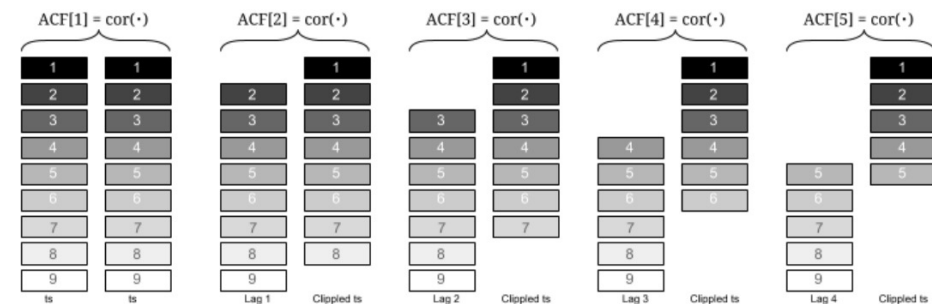
Partial ACF (PACF)

$$\text{PACF}(k) = \text{corr}(y_t, y_{t-k} \mid y_{t-1}, \dots, y_{t-k+1})$$

Measures the **direct** correlation at lag k , after removing the effect of all shorter lags.

Practical use: patterns in ACF/PACF suggest the order of the underlying process; we then confirm with AIC grid search.

ACF



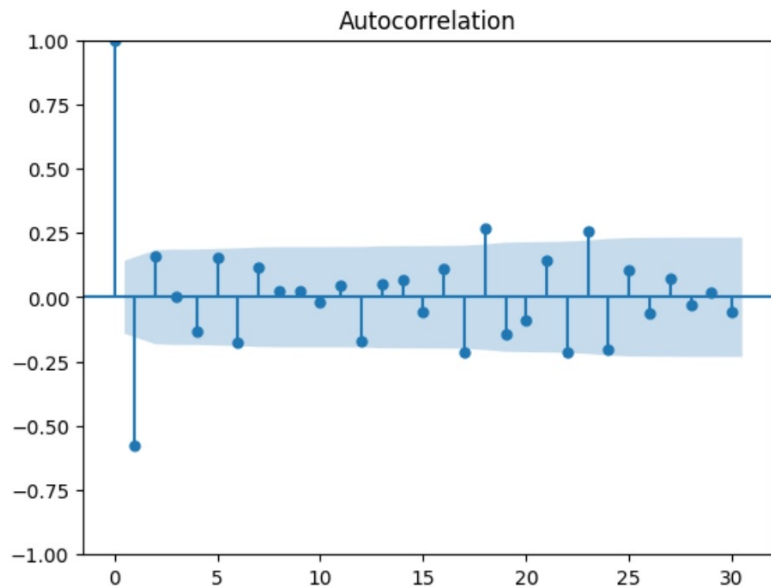
ACF at lag k : correlation between y_t and the same series shifted by k steps.

ACF cuts off at lag $q \Rightarrow MA(q)$

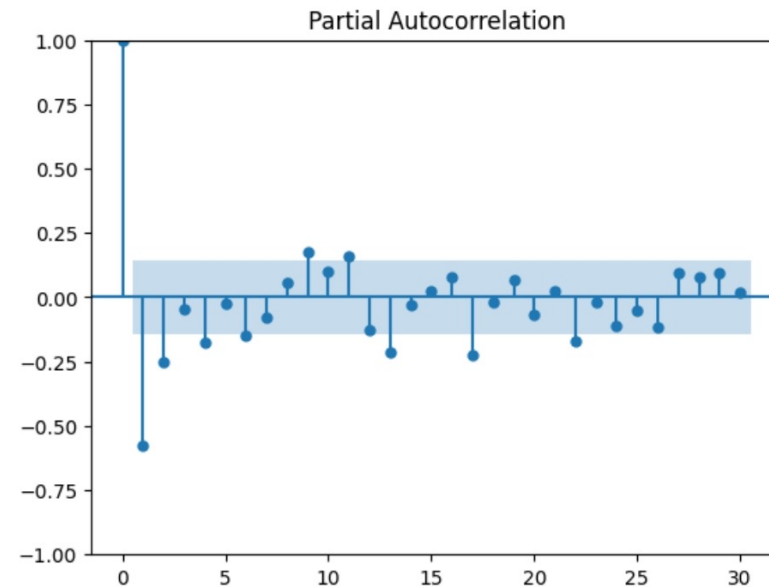
PACF cuts off at lag $p \Rightarrow AR(p)$

both tail off $\Rightarrow ARMA(p, q)$

ACF and PACF of the stationary series



ACF of y_t'' (after regular + seasonal differencing). The shaded band is the 95% confidence interval; spikes inside are not significant.



PACF of y_t'' . Both ACF and PACF show a strong spike at lag 1 and tail off, consistent with an ARMA process.

The lag scatter plots (not shown) confirm: **lag 1 has strong negative correlation** ($r = -0.58$); beyond lag 2, the series is mainly uncorrelated. This aligns with the MA(1) component found by AIC.

ARIMA: building blocks

$ARIMA(p, d, q)$ combines three ideas. The model operates on the **differenced** series y'_t (not the original).

AR(p)

current value depends
on the p most
recent **values**:

$$y'_t = \sum_{i=1}^p \phi_i y'_{t-i} + \varepsilon_t$$

I(d)

the series is
differenced d times
to achieve
stationarity

MA(q)

current value depends
on the q most
recent **forecast errors**:

$$y'_t = \varepsilon_t + \sum_{j=1}^q \theta_j \varepsilon_{t-j}$$

Example: ARIMA(1,1,1)

$$y'_t = c + \underbrace{\phi_1 y'_{t-1}}_{\text{AR term}} + \underbrace{\theta_1 \varepsilon_{t-1}}_{\text{MA term}} + \varepsilon_t, \quad y'_t = y_t - y_{t-1}$$

SARIMA: seasonal extension of ARIMA

$SARIMA(p, d, q)(P, D, Q)_m$ adds a **seasonal layer**:

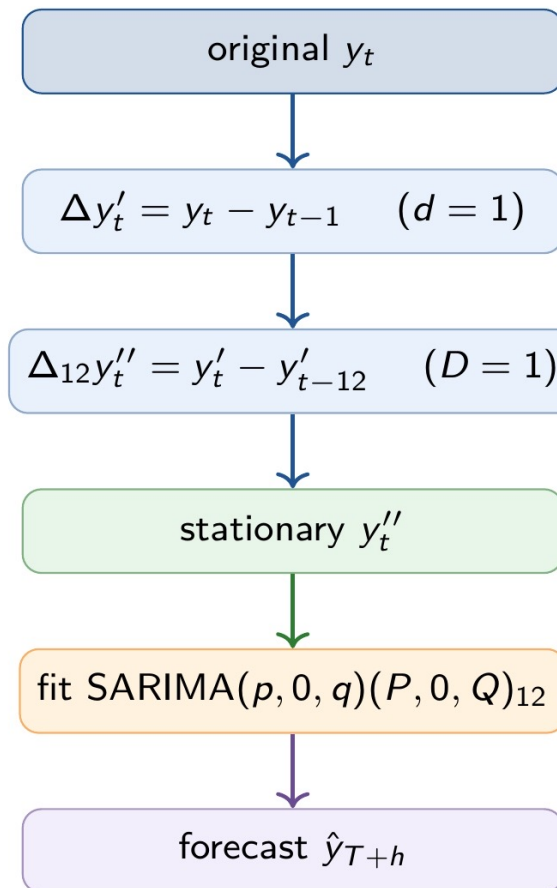
Parameter	Meaning
p, d, q	non-seasonal AR, I, MA orders
P, D, Q	seasonal AR, I, MA orders
m	seasonal period ($m = 12$, monthly)

The seasonal AR/MA terms capture dependencies at **multiples of m** : e.g. y_{t-12} , y_{t-24} , etc.

Our model

$SARIMA(3, 1, 1)(1, 1, 3)_{12}$

$d = 1$, $D = 1$ from ADF test; $m = 12$ (monthly). p, q, P, Q from AIC grid search.



Model selection: AIC

Given d, D, m fixed by the ADF test, we must choose p, q, P, Q .

The **Akaike Information Criterion** trades off goodness of fit against model complexity:

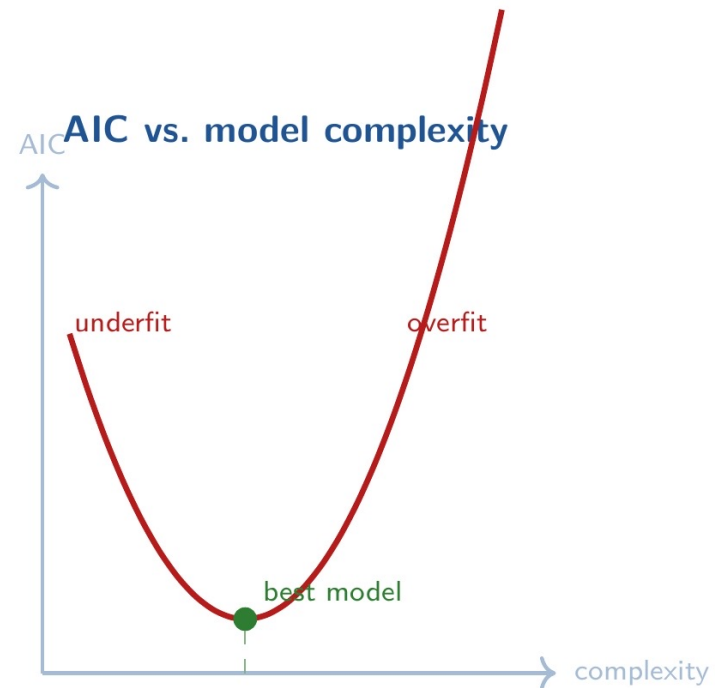
$$\text{AIC} = 2k - 2 \ln \hat{L}$$

k = number of parameters, $\hat{L}$ = maximized likelihood.

Lower AIC $\Rightarrow$ better model. The penalty $2k$ prevents overfitting.

Grid search

$p, q, P, Q \in \{1, 2, 3\} \Rightarrow 3^4 = 81$ combinations.
Best: $\text{SARIMA}(3, 1, 1)(1, 1, 3)_{12}$.



Underfitting: model too simple, high bias.
Overfitting: model fits noise, high variance.
AIC finds the sweet spot.

Model diagnostics: what are residuals?

After fitting SARIMA, we compute **residuals**:

$$e_t = y_t - \hat{y}_t$$

A well-specified model should have residuals that behave like **white noise**: independent, zero-mean, constant variance, approximately normal.

4 visual checks (plot_diagnostics)

- **Standardized residuals**: no pattern, no trend
- **Histogram + KDE**: should match $\mathcal{N}(0,1)$
- **QQ-plot**: quantiles should lie on the diagonal
- **Correlogram (ACF)**: no significant spikes at any lag

Ljung-Box test (quantitative)

Tests the null hypothesis that residuals up to lag H are **jointly uncorrelated**.

H_0 : residuals are independent

If **all** p -values > 0.05 : we cannot reject $H_0 \Rightarrow$ residuals look like white noise $\Rightarrow$ **model passes**.

Forecasting: in-sample vs out-of-sample

Out-of-sample (OOS) forecast

Predicts values **beyond** the training period — the true test of a forecasting model.

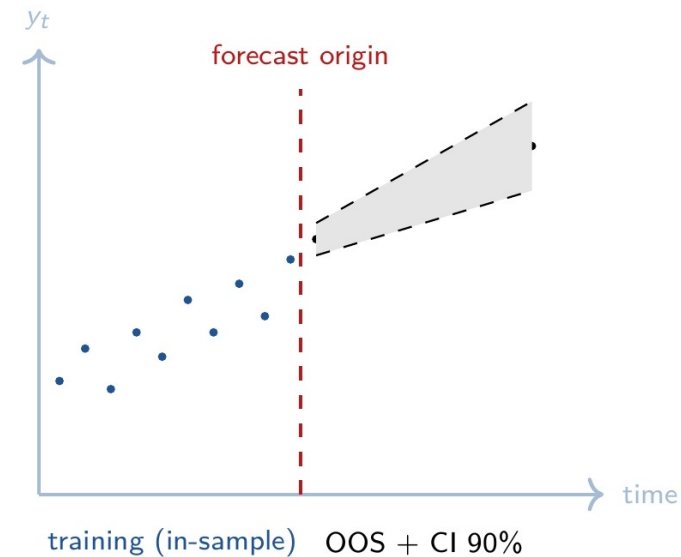
Methods: `forecast(steps=h)`,
`get_forecast(steps=h)`

In-sample prediction

Predicts values **within** the training period. Useful for checking fit and computing residuals.

Methods: `predict()`, `get_prediction()`

`get_forecast` and `get_prediction` return a `summary_frame` with **confidence intervals** at a chosen level (e.g. 90%).



Evaluation: MAPE and the seasonal naïve baseline

MAPE (Mean Absolute Percentage Error)

$$\text{MAPE} = \frac{1}{n} \sum_{t=1}^n \left| \frac{y_t - \hat{y}_t}{y_t} \right|$$

Scale-free, interpretable as average percentage error. Lower is better.

Seasonal naïve baseline

A simple reference: set each forecast equal to the value observed **one season ago**:

$$\hat{y}_{T+h} = y_{T+h-m(k+1)}$$

m : seasonal period, $k = \lfloor (h-1)/m \rfloor$.

For monthly data: forecast Feb. 2007 = Feb. 2006.

A good model should beat the naïve baseline. Otherwise, there is no point in the added complexity.

Result

Model	MAPE
Seasonal naïve	higher
SARIMA(3, 1, 1)(1, 1, 3) ₁₂	lower

SARIMA outperforms the naïve baseline: it captures the trend growth that the naïve method misses.

Why a single train/test split is not enough

A single split gives **one** error estimate over **one** test window.



single split — only one error estimate

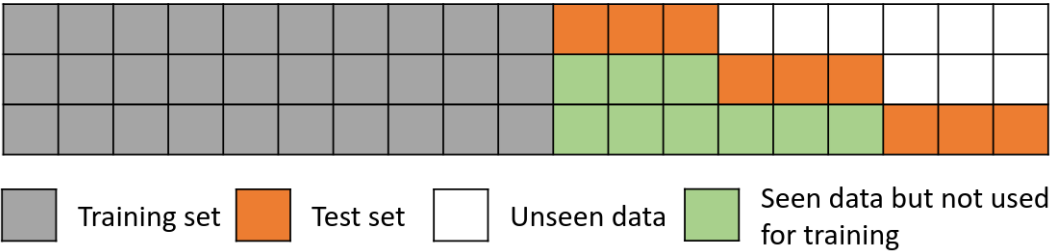
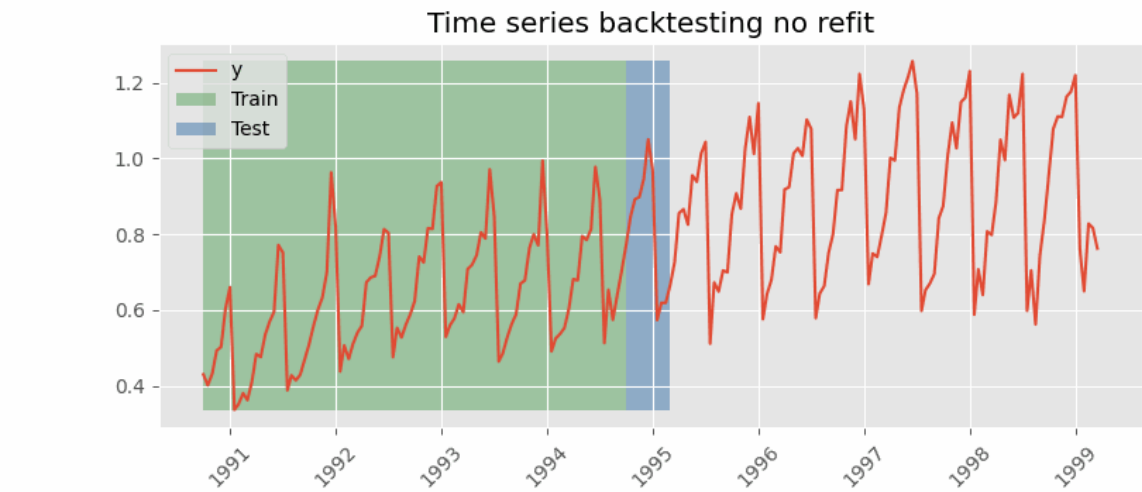
Problems

- High variance: the estimate depends heavily on which window you chose.
- Cannot measure how error **grows with the forecast horizon h** .
- One unlucky or lucky test period distorts the result.

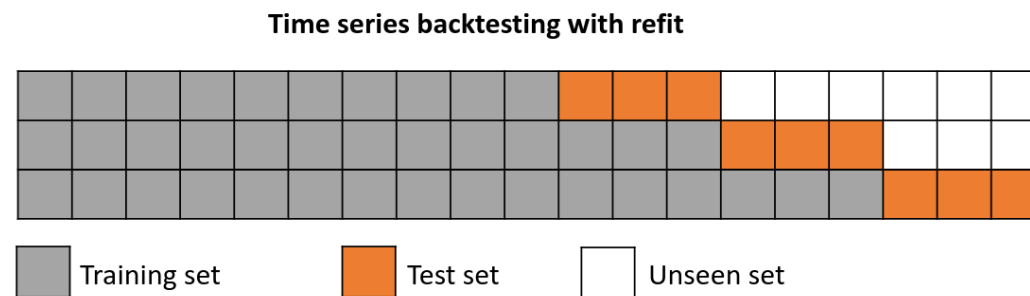
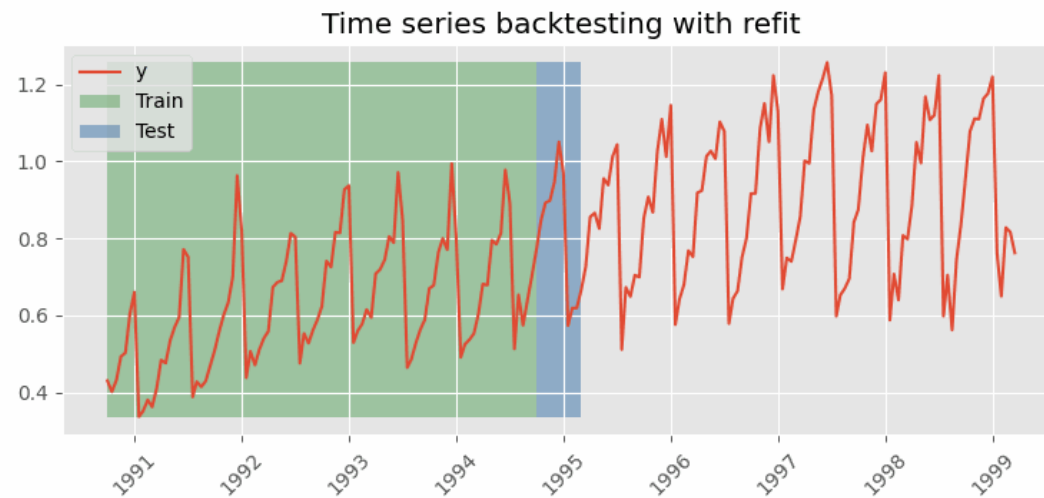
This single estimate may not be representative of real-world performance at different points in time or at different forecast horizons.

Solution: evaluate the model on **many** different splits — this is **backtesting** (= time series cross-validation).

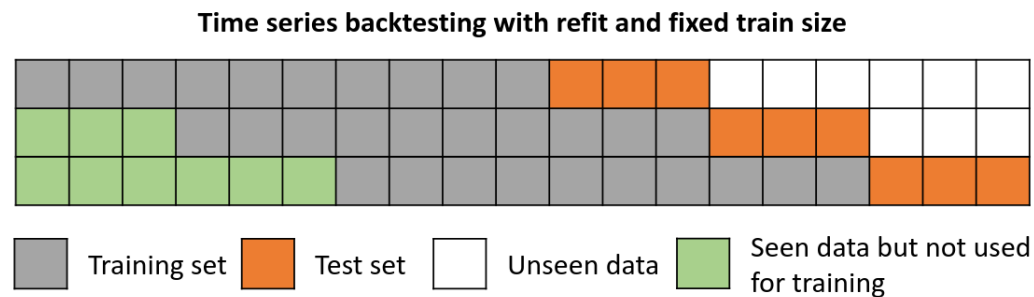
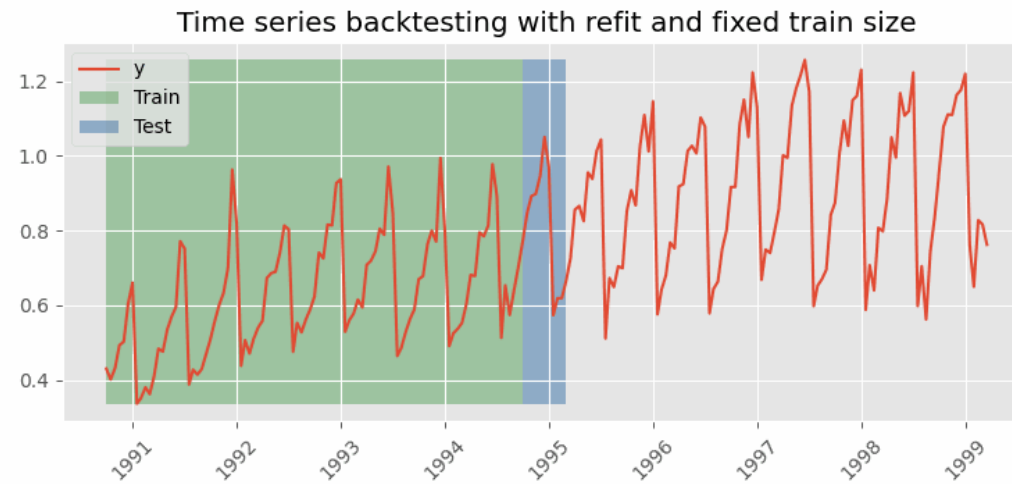
Backtesting



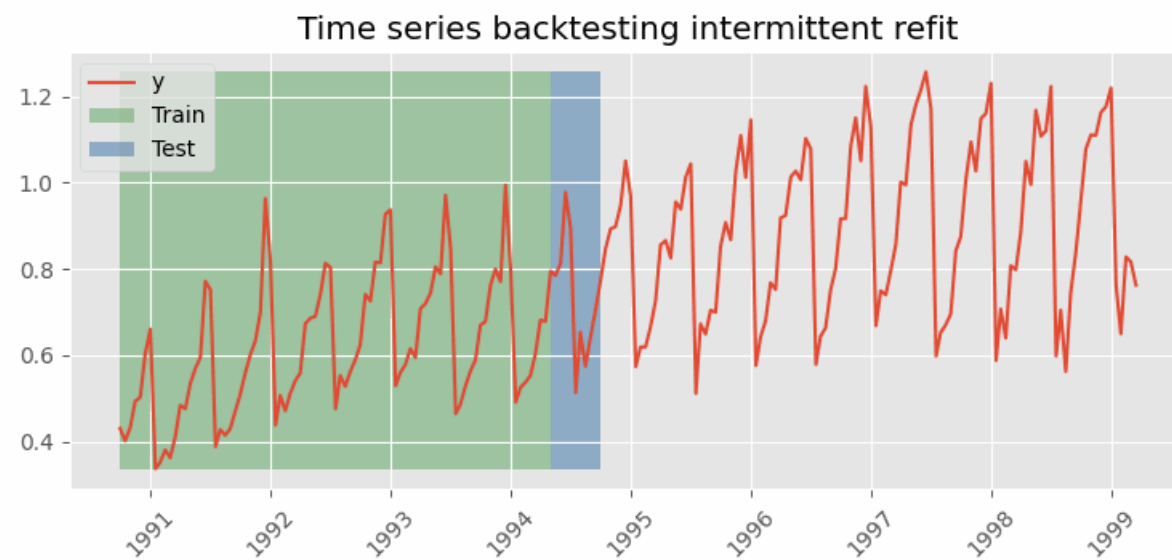
Backtesting



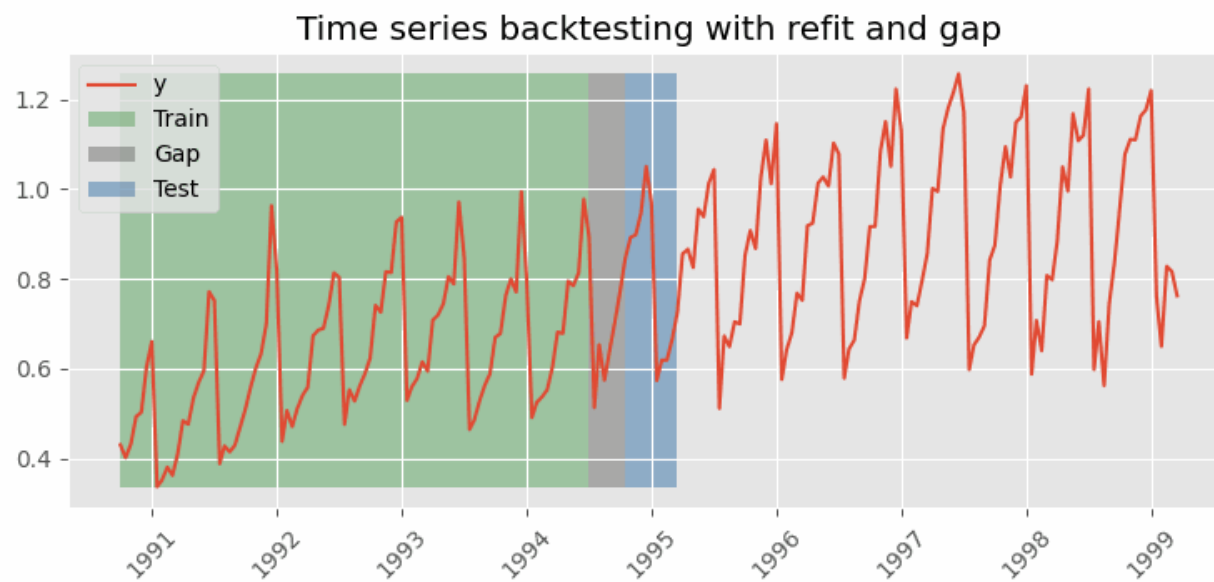
Backtesting



Backtesting



Backtesting



Backtesting: error by forecast horizon

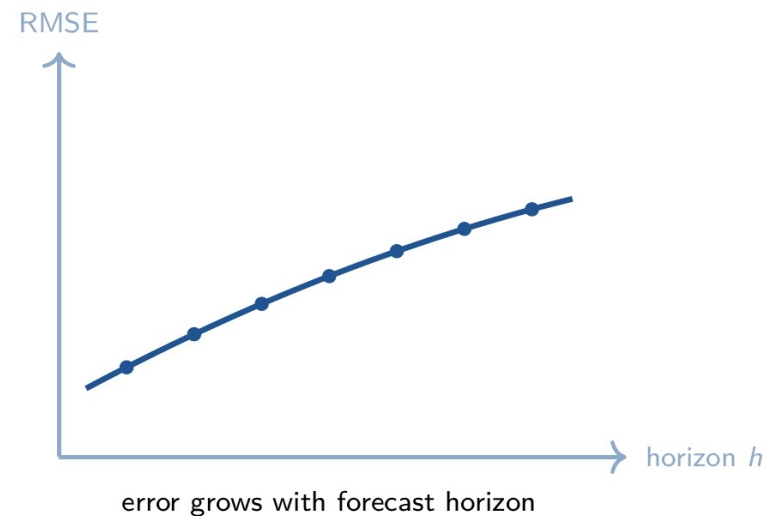
Backtesting produces many forecasts indexed by **horizon** h (how many steps ahead).

We **flatten** errors by horizon and compute RMSE for each h :

$$\text{RMSE}(h) = \sqrt{\frac{1}{N} \sum_i (y_{T_i+h} - \hat{y}_{T_i+h})^2}$$

What it tells you

How fast does performance degrade as you forecast further ahead? A good model should have slowly growing RMSE with h .



Typically, RMSE grows with h because uncertainty accumulates. This plot reveals at which horizon the model becomes unreliable.

ML for forecasting: the lag feature idea

Any sklearn **regression model** can forecast a time series by transforming it into a supervised learning problem.

Key idea: use the p most recent observations as input features to predict the next value.

Lag feature matrix ($p = 3$ lags)

$$\underbrace{\begin{pmatrix} y_1 & y_2 & y_3 \\ y_2 & y_3 & y_4 \\ y_3 & y_4 & y_5 \\ \vdots & \vdots & \vdots \end{pmatrix}}_{\text{features } X} \rightarrow \underbrace{\begin{pmatrix} y_4 \\ y_5 \\ y_6 \\ \vdots \end{pmatrix}}_{\text{target } y}$$

Each row is an independent training sample. You can use any sklearn regressor (LinearRegression, RandomForest, etc.)

skforecast automates this with ForecasterAutoreg. It handles:

- building the lag matrix from y
- recursive multi-step forecasting (feeding predictions as inputs for longer horizons)
- adding exogenous variables as extra columns

Equivalence

With LinearRegression() and p lags, ForecasterAutoreg is essentially fitting an **AR(p)** model — but you can swap in any regressor.

Aside: encoding cyclic time features

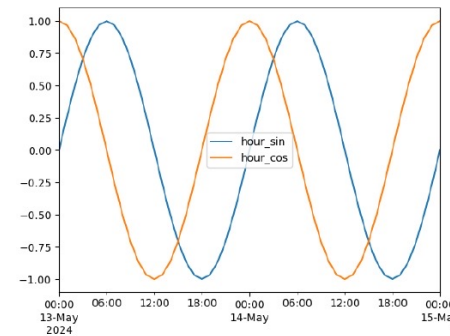
When using ML models, time features like *hour of day* or *month of year* are **cyclic**: hour 23 and hour 0 are adjacent, yet numerically far apart.

Using raw integers would mislead the model into thinking 23 and 0 are far from each other.

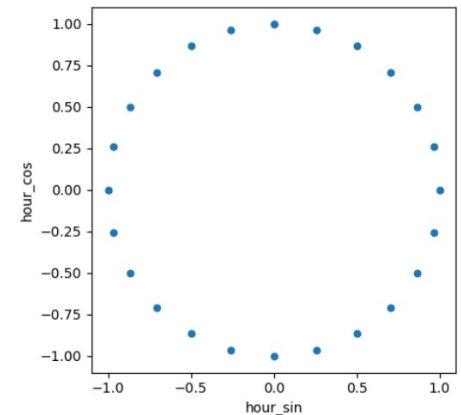
Solution: encode as a point on the unit circle:

$$x_{\sin} = \sin\left(\frac{2\pi h}{24}\right), \quad x_{\cos} = \cos\left(\frac{2\pi h}{24}\right)$$

Now hours 23 and 0 are adjacent on the circle.



sin / cos are smooth
periodic functions.



The 24 hours lie on a
circle — 23h and 0h are
adjacent.

Part 2

Notebook walkthrough

Setup: imports

```
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
from statsmodels.tsa.seasonal import seasonal_decompose, STL
from statsmodels.stats.diagnostic import acorr_ljungbox
from statsmodels.tsa.statespace.sarimax import SARIMAX
from statsmodels.tsa.stattools import adfuller
from itertools import product
from sklearn.metrics import mean_absolute_percentage_error as mape
from skforecast.ForecasterAutoreg import ForecasterAutoreg
import pandas as pd, numpy as np, matplotlib.pyplot as plt
```

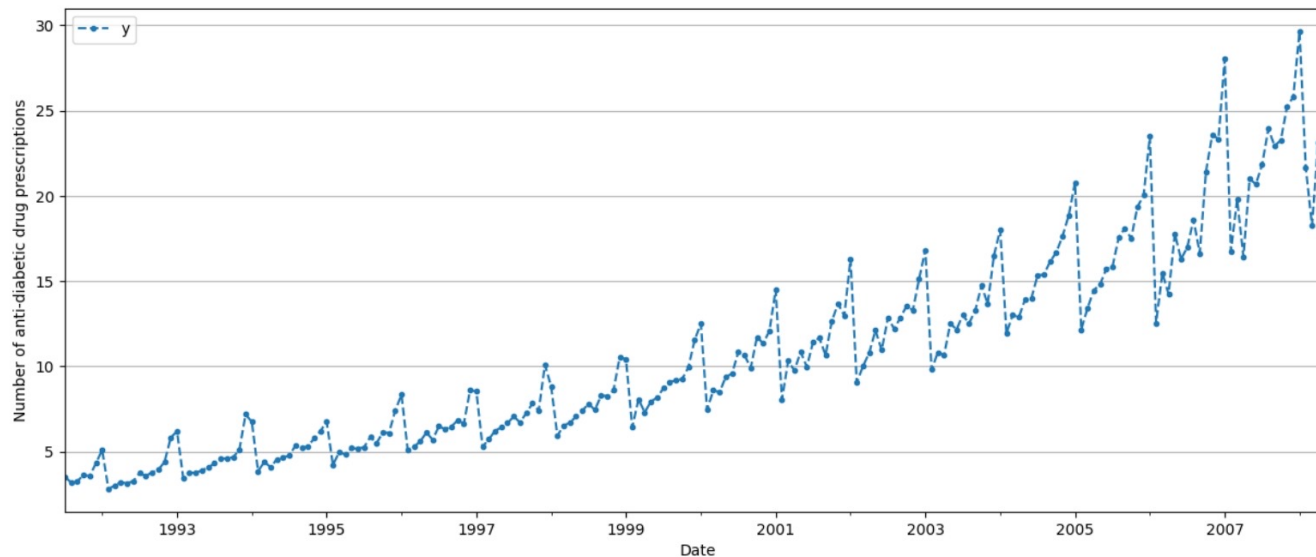
statsmodels — SARIMA, decomposition, statistical tests

sklearn — MAPE, regressors

skforecast — ML forecasting wrapper.

Load and visualize the time series

```
df = pd.read_csv('./dataset/AusAntidiabeticDrug.csv')
df['ds'] = pd.to_datetime(df['ds'], format='%Y-%m-%d')
df = df.set_index('ds').asfreq('MS').sort_index()
```

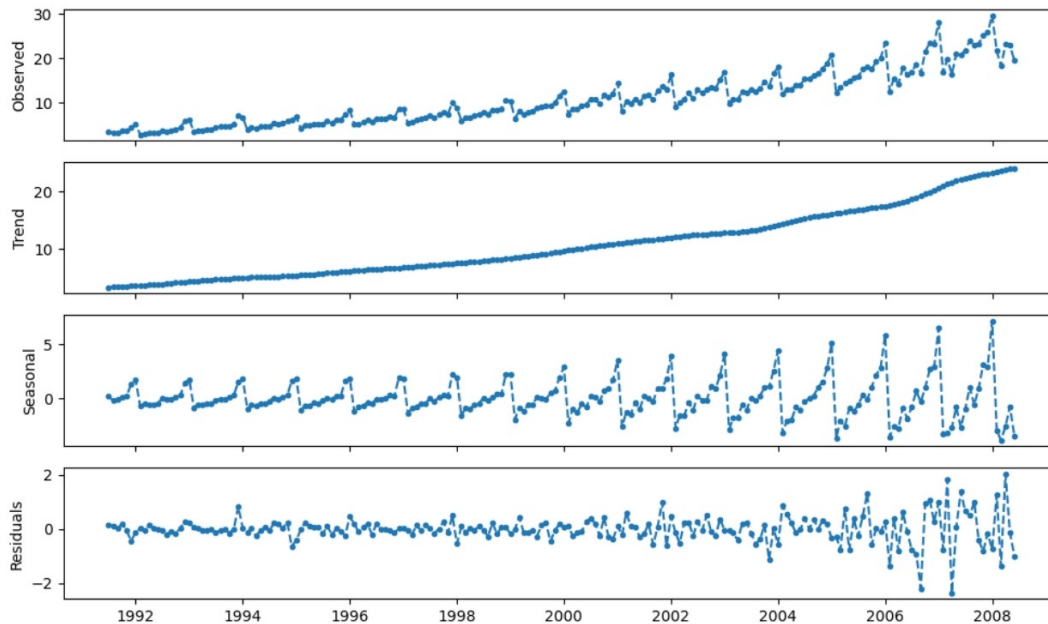


`asfreq('MS')` sets the index frequency to *Month Start* — required by statsmodels.

The plot confirms **trend** (long-term growth) and **seasonality** (annual oscillation with growing amplitude).

STL decomposition

```
decomposition = STL(df.y, period=12).fit()
fig, (ax1, ax2, ax3, ax4) = plt.subplots(4, 1, sharex=True, figsize=(10,6))
ax1.plot(decomposition.observed, '.--'); ax1.set_ylabel('Observed')
ax2.plot(decomposition.trend, '.--'); ax2.set_ylabel('Trend')
ax3.plot(decomposition.seasonal, '.--'); ax3.set_ylabel('Seasonal')
ax4.plot(decomposition.resid, '.--'); ax4.set_ylabel('Residuals')
```



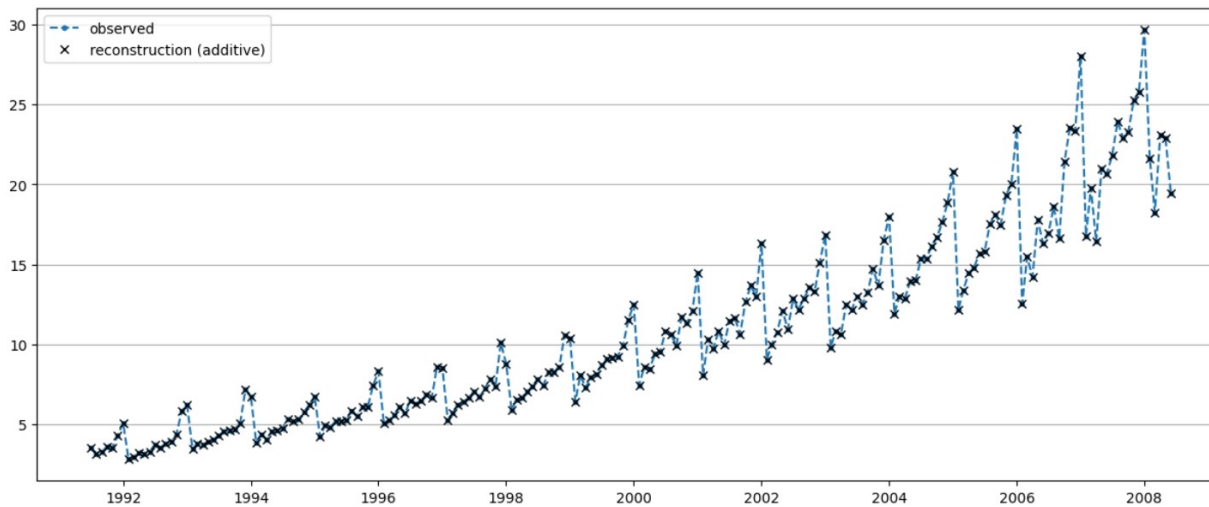
Trend: smooth growth confirms non-stationarity.

Seasonal: the annual pattern repeats, but its amplitude grows — sign that **multiplicative** decomposition would be more natural.

Residuals: mostly small, but their spread increases after 2006 — the model leaves some structure unexplained.

Additive reconstruction check

```
plt.plot(decomposition.observed, '--', label='observed')  
plt.plot(decomposition.resid + decomposition.trend + decomposition.seasonal,  
         'xk', label='reconstruction (additive)')
```

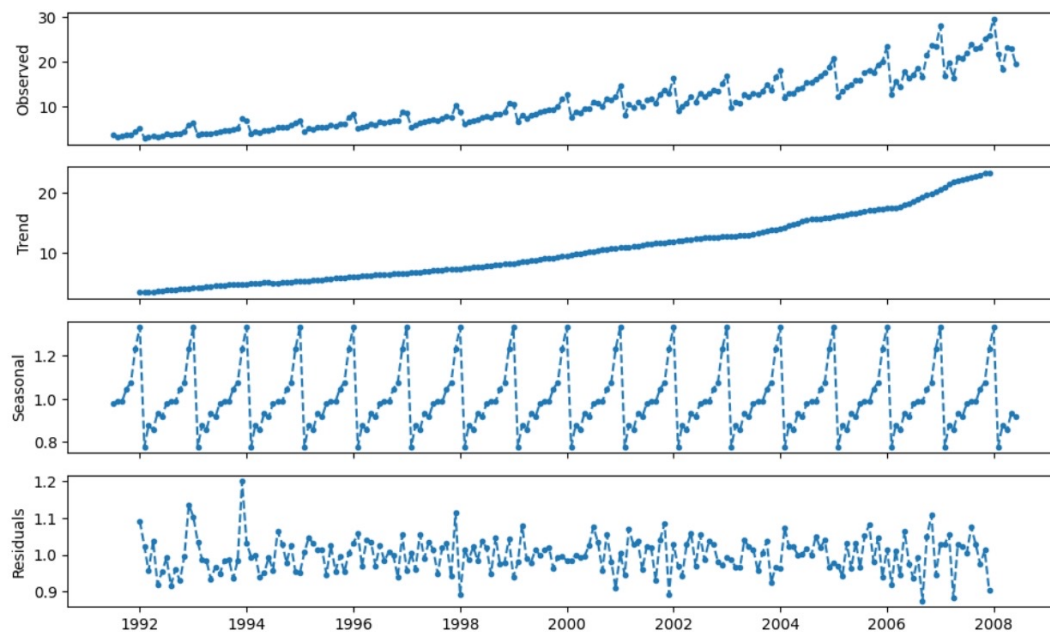


The crosses ($\times$) lie exactly on the original dashed line: $t_t + s_t + \varepsilon_t = x_t$ for every point.

This verifies the decomposition is **exact** and consistent — the three components together account for 100% of the signal.

Multiplicative decomposition

```
decompose_m = seasonal_decompose(df.y, period=12, model='multiplicative')
```



The **seasonal** component now oscillates around **1.0** (a ratio).

The **residuals** also oscillate around 1.0 and are **much more homogeneous** in spread — confirming this model is a better fit for this series.

Comparison: in the additive plot, residuals grow larger after 2004; here they stay stable throughout. Multiplicative wins.

Testing stationarity and differencing

```
# Original: p-value = 1.0 -> not stationary
ad_fuller_result = adfuller(df.y)

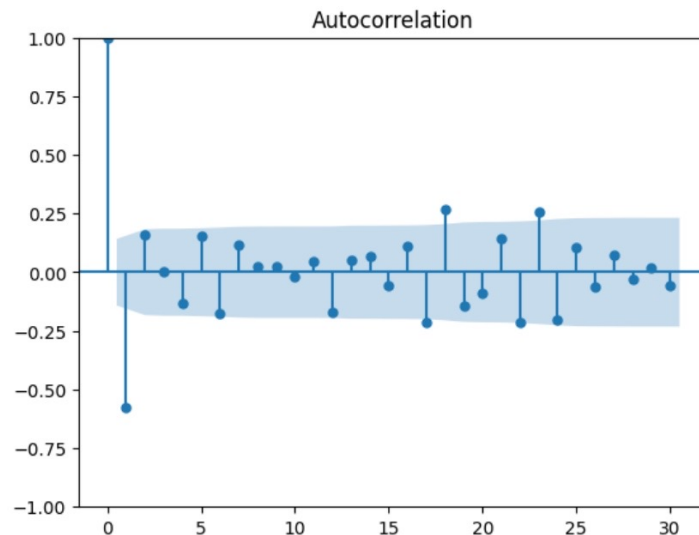
# First-order difference: p-value = 0.12 -> still not stationary
y_diff = np.diff(df.y, n=1)
ad_fuller_result = adfuller(y_diff)

# First + seasonal difference (m=12): p << 0.05 -> stationary!
y_diff_seasonal_diff = y_diff[12:] - y_diff[0:-12]
ad_fuller_result = adfuller(y_diff_seasonal_diff)
print(f'p-value: {ad_fuller_result[1]}')
```

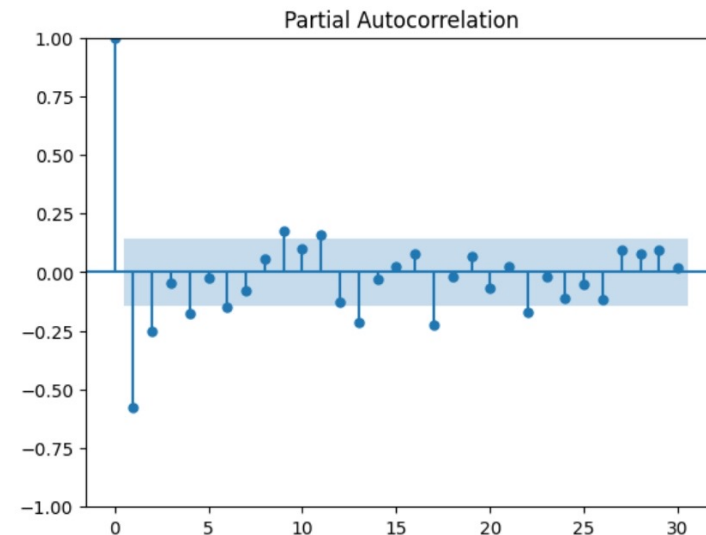
Only after applying both differences does the ADF test confirm stationarity. This fixes $d = 1$ and $D = 1$ in the SARIMA model.

ACF and PACF of the stationary series

```
plot_acf(y_diff_seasonal_diff, lags=30)
plot_pacf(y_diff_seasonal_diff, lags=30)
```



ACF: strong spike at lag 1 (negative),
then all inside the CI band.

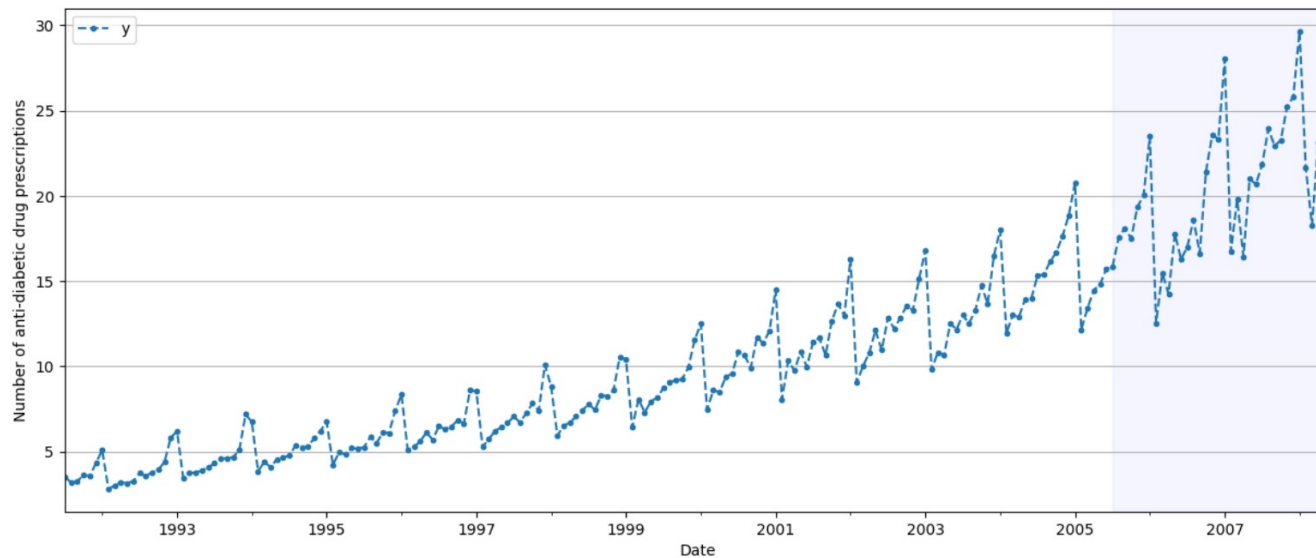


PACF: same pattern. Both cut off quickly,
consistent with MA(1) or AR(1) structure.

The shaded band is the 95% confidence interval. Spikes inside it are not significantly different from zero. The strong lag-1 spike guides the grid search range for p and q .

Train/test split

```
steps = 36  
train = df[:-steps]  
test = df[-steps:]
```



The blue shaded area marks the **test set** (last 36 months).

No shuffling: time order must be respected to avoid data leakage from the future.

SARIMA: grid search and fitting

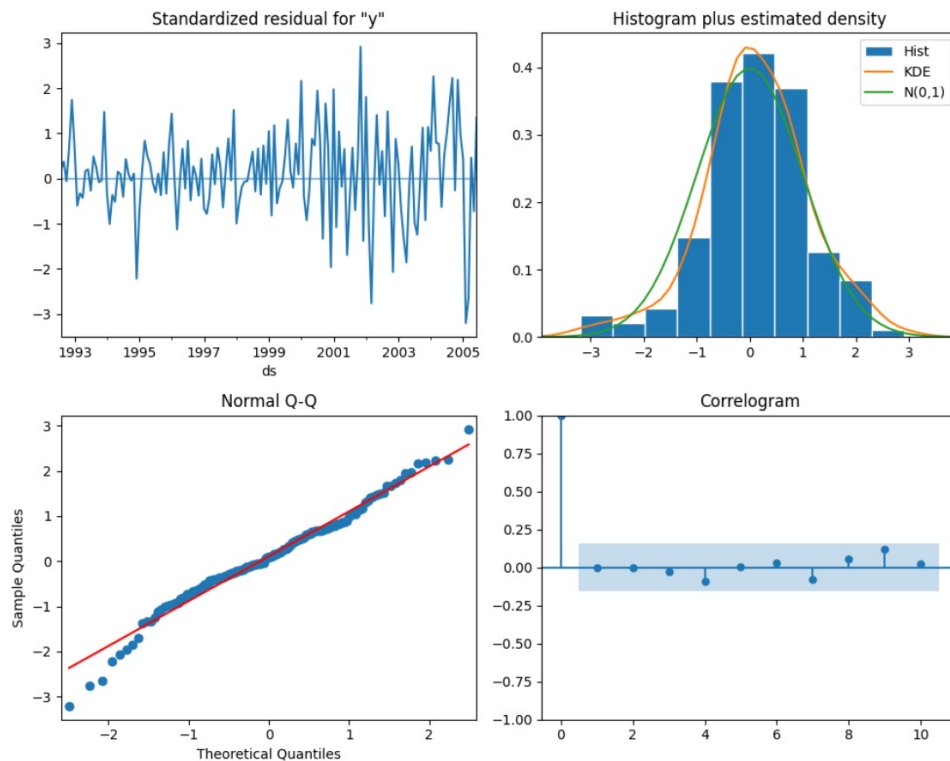
```
from optimizer import optimize_SARIMAX
ps=range(1,4); qs=range(1,4); Ps=range(1,4); Qs=range(1,4)
order_list = list(product(ps, qs, Ps, Qs)) # 81 combinations
SARIMA_result_df = optimize_SARIMAX(train, None, order_list, d=1, D=1, s=12)

# Best: p=3, q=1, P=1, Q=3
SARIMA_model = SARIMAX(train, order=(3,1,1), seasonal_order=(1,1,3,12))
SARIMA_model_fit = SARIMA_model.fit(dispatch=False)
print(SARIMA_model_fit.summary())
```

`optimize_SARIMAX` fits all 81 configurations and returns a DataFrame sorted by AIC. The winning combination $SARIMA(3,1,1)(1,1,3)_{12}$ minimizes AIC. The `summary()` prints coefficients, standard errors, and information criteria.

Residual diagnostics

```
SARIMA_model_fit.plot_diagnostics(figsize=(10,8))
df_lb = acorr_ljungbox(SARIMA_model_fit.resid.values, period=12)
print((df_lb.lb_pvalue < 0.05).sum())    # -> 0: all p-values above threshold
```



Standardized residuals: oscillate around 0, no trend.

Histogram + KDE: matches $\mathcal{N}(0, 1)$ well (green).

QQ-plot: points align on the diagonal — residuals are approximately normal.

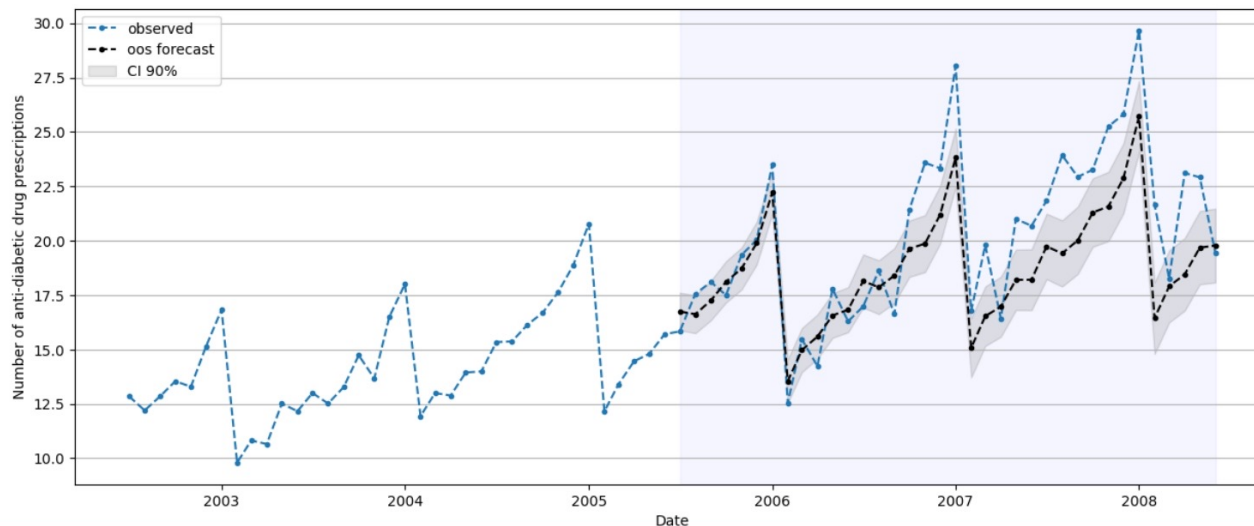
Correlogram: no spikes outside the CI band.

Ljung-Box: 0 p-values below 0.05. All checks pass — **model is valid for forecasting.**

Out-of-sample forecast: point + CI

```
fcast = SARIMA_model_fit.get_forecast(steps=36)
df_fcast = fcast.summary_frame(alpha=0.10) # 90% CI

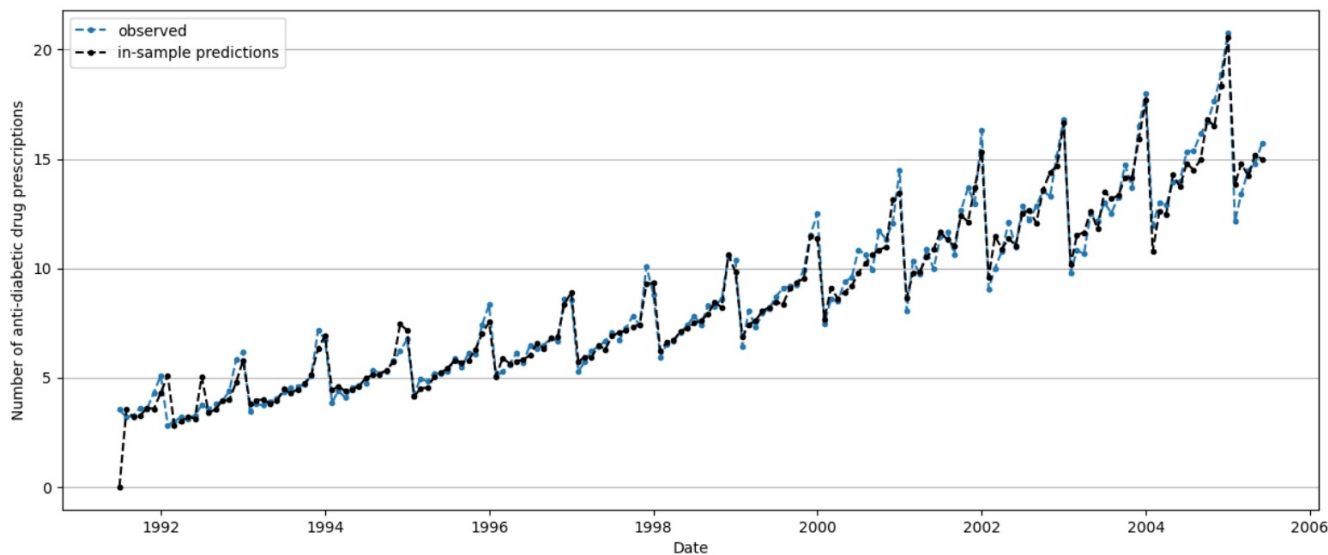
fig, ax = plt.subplots(figsize=(15,6))
ax.plot(df.y[-72:], '--', label='observed')
ax.plot(df_fcast['mean'], '.k--', label='oos forecast')
ax.fill_between(df_fcast.index,
               df_fcast['mean_ci_lower'], df_fcast['mean_ci_upper'],
               color='k', alpha=0.1, label='CI 90%')
```



The black forecast follows the observed values. The gray band is the 90% confidence interval — it **widens** as the horizon grows, reflecting accumulating uncertainty.

In-sample predictions

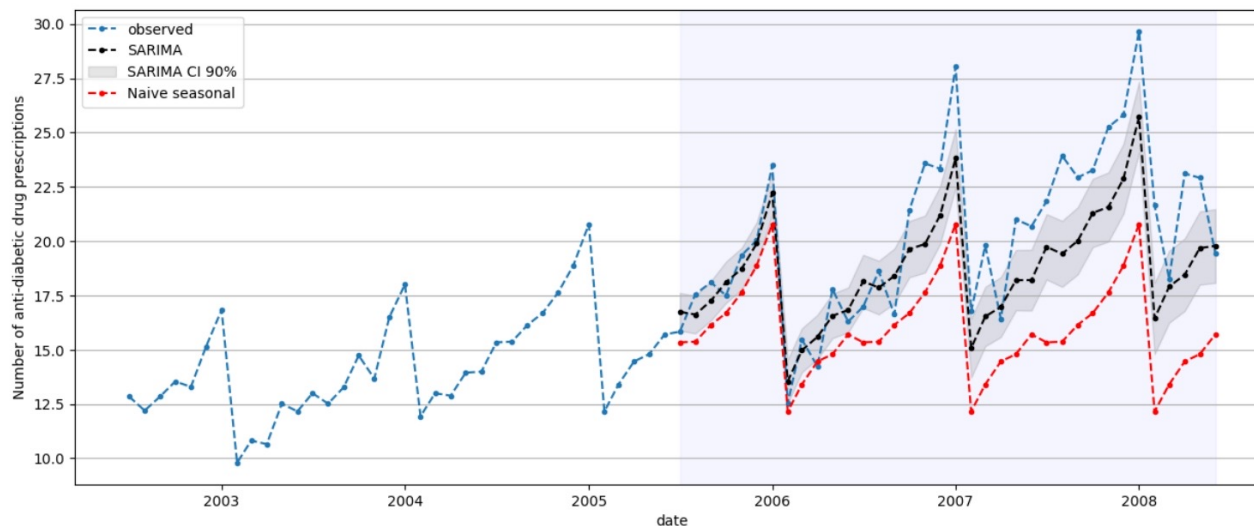
```
fig, ax = plt.subplots(figsize=(15,6))
ax.plot(train, '--', label='observed')
ax.plot(SARIMA_model_fit.predict(), '--k', label='in-sample predictions')
```



In-sample predictions track the observed values closely, confirming the model has learned the trend and seasonal pattern. The first few points are zero due to initialization of the AR component.

Evaluation: SARIMA vs seasonal naïve

```
pred_df['SARIMA'] = SARIMA_model_fit.forecast(steps=36)
last_season = df['y'][156:168].values
pred_df['naive_seasonal'] = [v for _ in range(3) for v in last_season]
mape_naive = mape(pred_df.y, pred_df.naive_seasonal)
mape_SARIMA = mape(pred_df.y, pred_df.SARIMA)
```



Naïve seasonal (red): repeats last year — misses the growing trend.

SARIMA (black): follows the growing trend, much closer to the observed values.

SARIMA achieves a **lower MAPE**, confirming it is a better model.

Backtesting: expanding window with refit

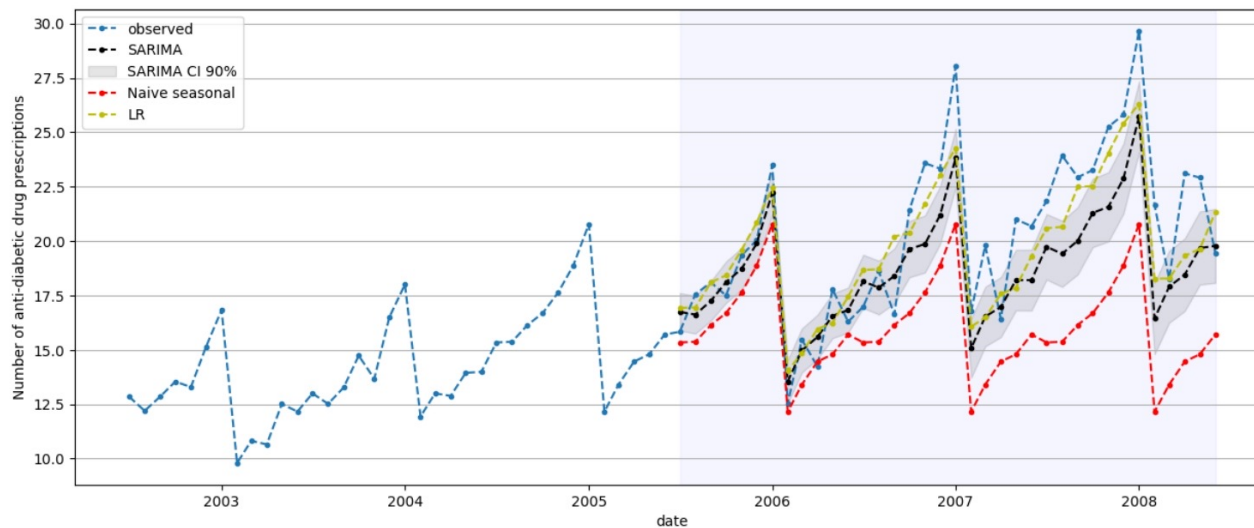
```
nforecasts = 12; n_init_training = 168; forecasts = {}
mod = SARIMAX(df.y.iloc[:n_init_training], order=(3,1,1), seasonal_order=(1,1,3,12))
res = mod.fit(dispatch=False)
for t in range(n_init_training, nobs):
    forecasts[t] = res.forecast(steps=nforecasts)
    res = res.append(df.y.iloc[t:t+1], refit=False) # expand without full refit

# Flatten errors by horizon
def flatten(col): return col.dropna().reset_index(drop=True)
flattened = forecast_errors.apply(flatten)
flattened.index = (flattened.index + 1).rename('horizon')
rmse = (flattened**2).mean(axis=1)**0.5
```

At each step, the model observes one new point and forecasts 12 steps ahead. Errors are indexed by **horizon** $h = 1, \dots, 12$ to reveal how forecast quality degrades.

ML forecasting: ForecasterAutoreg

```
forecaster = ForecasterAutoreg(regressor=LinearRegression(), lags=12)
forecaster.fit(y=train.y)
X, y_vec = forecaster.create_train_X_y(train.y)    # inspect lag matrix
predictions_LR = forecaster.predict(steps=36)
```

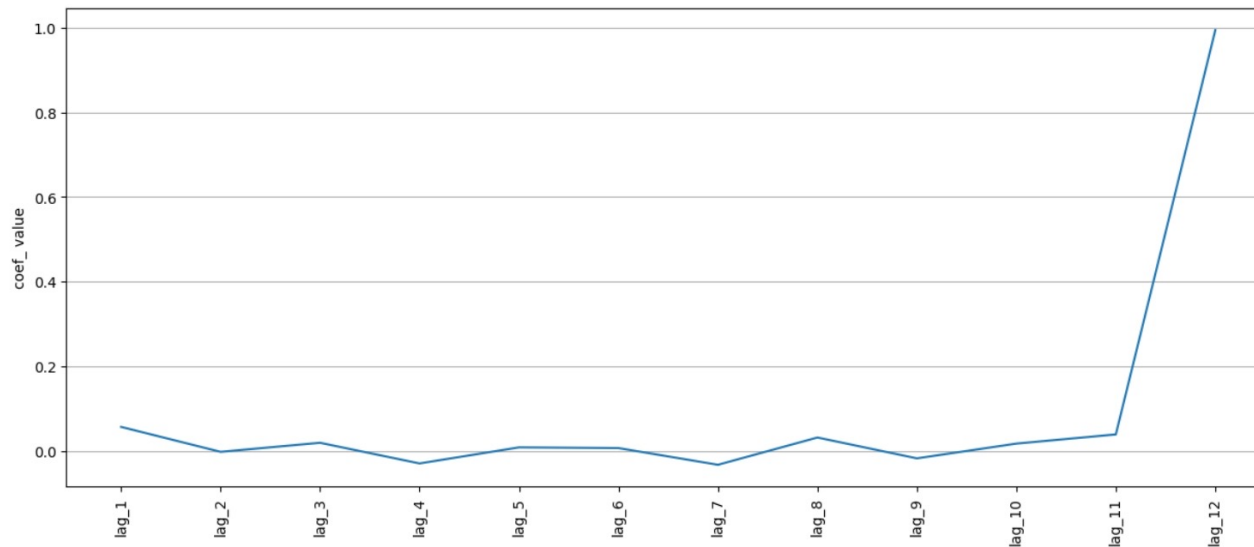


LR (yellow): linear regression with 12 lags. Performs comparably to SARIMA on this data, but without seasonal structure baked in.

SARIMA (black) and **LR** both beat the naïve seasonal (red) baseline.

Feature importance: which lags matter?

```
feature_importance = forecaster.get_feature_importances()  
feature_importance['importance'].plot()
```



Lag 12 dominates: its coefficient is by far the largest. This makes sense — monthly data has a period of 12, so *the same month last year* is the most informative predictor.

Lag 1 has a small positive effect; all other lags are nearly zero. The LinearRegression model effectively recovers the seasonal structure.

Summary

Concepts:

- **Decomposition:** additive vs multiplicative; STL
- **Stationarity:** ADF test; 1st order and seasonal differencing
- **Temporal correlation:** ACF and PACF
- **ARIMA** building blocks: AR, I, MA
- **SARIMA:** seasonal AR and MA terms; parameters fixed or selected by AIC
- **Residual diagnostics:** visual + Ljung-Box
- **Prediction:** In-sample vs out-of-sample
- **Evaluation:** MAPE, seasonal naïve baseline
- **Backtesting:** expanding/fixed window, error by horizon
- **Nested cross-validation** for unbiased evaluation + model selection

To remember

- Always **check stationarity** before modeling
- AIC selects SARIMA orders without overfitting
- Validate residuals before trusting forecasts
- A single train/test split is fragile; use backtesting
- Any sklearn regressor becomes a forecaster with `skforecast`
- Feature importance confirms seasonality

References

Peixeiro, M. (2022). *Time Series Forecasting in Python*. Manning.

Hyndman & Athanasopoulos (2021). *Forecasting: Principles and Practice*, 3rd ed. OTexts.

Amat Rodrigo & Escobar Ortiz. *skforecast* v0.6.0. CC-BY 4.0.